PATRÓN DE DISEÑO

Template Method

Preparación de café y té en Java

Integrantes:	Marcelo E. Mansilla, Adrian Castaño
Materia:	Algoritmos y Estructuras de datos II
Docente:	Martin Fidel Santos Silva
Fecha	28/10/2026

Informe, comparación de implementaciones y guía de ejecución

1. Problema que se busca resolver

En una aplicación sencilla de cafetería se deben preparar distintas bebidas calientes, por ejemplo café y té. Aunque cada bebida tiene pasos particulares, ambas comparten una secuencia general: mostrar el inicio, calentar el agua, preparar la base, servir, añadir complementos y mostrar un resumen.

Cuando cada clase implementa el proceso completo por separado, la misma secuencia queda duplicada. Si se cambia el orden de los pasos o se agrega una etapa común, hay que modificar cada clase y comprobar que ambas sigan comportándose de la misma manera. Esta duplicación aumenta el mantenimiento y facilita que las implementaciones se desincronicen.

Ejemplo simplificado sin patrón

```
class PreparadorCafe {
    void preparar() {
        mostrarInicio();
        hervirAgua();
        colocarCafeEnFiltro();
        servirEnTaza();
        agregarAzucar();
        mostrarResumen();
    }
}

class PreparadorTe {
    void preparar() {
        mostrarInicio();
        hervirAgua();
        dejarTeEnInfusion();
        servirEnTaza();
        agregarAzucar();
        mostrarResumen();
    }
}
```

Los métodos para preparar la base son distintos, pero el resto del algoritmo se repite. Además, una futura bebida —por ejemplo chocolate caliente— tendría que volver a copiar la secuencia general.

Requisitos de la solución

• Mantener un orden de preparación común y visible.
• Permitir que cada bebida defina su preparación específica.
• Compartir los pasos que son iguales para todas las bebidas.
• Poder agregar una bebida nueva sin duplicar el algoritmo completo.

Objetivo: centralizar el esqueleto del algoritmo en una clase base y delegar los pasos variables a las subclases.

2. Por qué elegir Template Method

Template Method define en una clase base la estructura de un algoritmo y permite que las subclases implementen ciertos pasos sin cambiar el orden general. En este ejemplo, **preparar()** fija la secuencia completa; **prepararBase()** y **nombreBebida()** representan las partes que varían entre café y té. El método **agregarComplementos()** contiene el comportamiento común y puede ampliarse en las clases concretas.

Comparación con Strategy

Criterio	Template Method	Strategy
Idea central	La clase base define el orden; las subclases completan pasos.	Un objeto delega una parte del comportamiento a una estrategia intercambiable.
Variación	Se logra principalmente mediante herencia y redefinición de métodos.	Se logra mediante composición e intercambio de objetos estrategia.
Flujo del algoritmo	La secuencia general queda centralizada y protegida.	El contexto coordina el uso de la estrategia; el flujo puede ser más flexible.
Ventaja	Evita duplicar el esqueleto del proceso.	Permite cambiar el comportamiento en tiempo de ejecución.
Limitación	La herencia puede hacer rígida la estructura y acoplar las subclases a la clase base.	Puede requerir más clases y delegación explícita.
Adecuado cuando…	Las variantes comparten el mismo orden de pasos.	El comportamiento debe poder intercambiarse sin cambiar la clase de contexto.

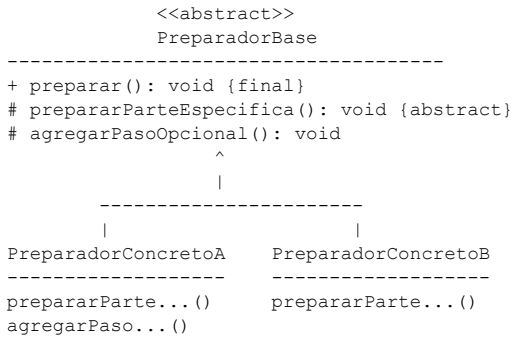
Se elige Template Method porque el orden de preparación no debería cambiar entre bebidas. Strategy sería preferible si el usuario necesitara combinar o intercambiar algoritmos de preparación en tiempo de ejecución, sin depender de una jerarquía de clases.

Alternativas consideradas

Una función condicional central podría resolver el ejemplo, pero concentraría las diferencias en bloques if/else que crecerían con cada bebida. La duplicación sin patrón es simple al comienzo, pero se vuelve costosa cuando cambian los pasos comunes. Template Method ofrece un punto intermedio: estructura compartida y variaciones localizadas.

3. UML genérico

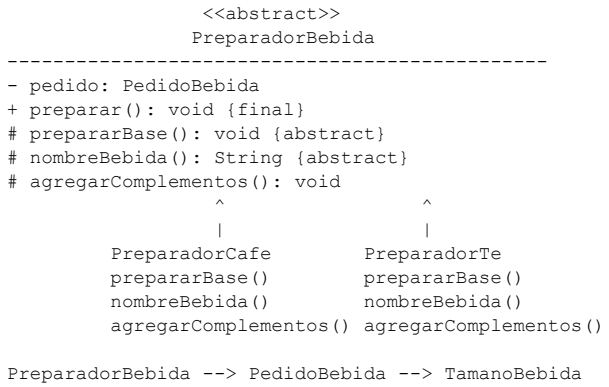
La clase abstracta define el método plantilla **preparar()** y un paso abstracto que deben completar las subclases. El paso opcional tiene una implementación común y solo se redefine cuando una variante necesita ampliar el comportamiento.



Responsabilidades

- **Clase base:** contiene el algoritmo completo y define qué pasos son comunes o variables.
- **Método plantilla:** método público que fija el orden del proceso.
- **Operación abstracta:** obliga a las subclases a implementar una parte necesaria.
- **Paso opcional:** ofrece un comportamiento predeterminado que puede ampliarse.

4. UML del ejemplo Java



La clase **PedidoBebida** encapsula el tamaño elegido y la cantidad de azúcar. **TamanoBebida** representa las opciones chica, mediana y grande, junto con el volumen de agua correspondiente.

5. Implementación en Java

La implementación con patrón separa el algoritmo común de las diferencias entre bebidas. El método **preparar()** es final para evitar que una subclase cambie accidentalmente la secuencia.

Clase base: fragmento representativo

```
public final void preparar() {
    mostrarInicio();
    hervirAgua();
    prepararBase();
    servirEnTaza();
    agregarComplementos();
    mostrarResumen();
}

protected abstract void prepararBase();
protected abstract String nombreBebida();

protected void agregarComplementos() {
    if (pedido.getCucharaditasAzucar() == 0) {
        System.out.println("Sin azucar agregada");
    } else {
        System.out.println("Agregando azucar");
    }
}
```

Clase concreta: café

```
@Override
protected void prepararBase() {
    System.out.println("Colocando cafe molido en el filtro");
    System.out.println("Pasando agua caliente por el cafe");
}

@Override
protected void agregarComplementos() {
    super.agregarComplementos();
    System.out.println("Espolvoreando una pizca de canela");
}
```

La clase de té implementa su propia base —dejar las hojas o la bolsita en infusión— y amplía los complementos con limón. Ambas reutilizan el método plantilla.

Ejecución y comparación

El proyecto incluye dos versiones ejecutables: **java/sin_patron** y **java/con_patron**. ; la diferencia importante está en la organización del código, no en el resultado final.

6. Uso real y cuándo no conviene

El patrón resulta útil cuando un proceso tiene una estructura estable y varios pasos que cambian según el tipo concreto. Algunos ejemplos habituales son flujos de procesamiento de archivos, etapas de validación y transformación, preparación de documentos y procesos de compilación. En todos los casos conviene confirmar que el orden general es verdaderamente común.

Cuándo conviene

• Hay un algoritmo general que debe seguir siempre el mismo orden.
• Varias implementaciones comparten buena parte de los pasos.
• Se quiere evitar que cada clase copie el proceso completo.
• Las diferencias pueden expresarse como métodos redefinibles de una clase base.

Cuándo no conviene

• Las variantes necesitan órdenes de ejecución muy diferentes.
• Se requiere cambiar algoritmos dinámicamente durante la ejecución.
• La herencia no encaja con el modelo del dominio o produce una jerarquía difícil de mantener.
• Hay tantas excepciones a la secuencia común que la clase base deja de representar un flujo estable.

Extensión con chocolate caliente

Para agregar chocolate, se crearía una subclase **PreparadorChocolate** que implemente **nombreBebida()** y **prepararBase()**. Si necesita un complemento especial, puede ampliar **agregarComplementos()** y llamar primero a **super.agregarComplementos()** para conservar el azúcar común. No es necesario copiar el método **preparar()**.

7. Conclusiones

Template Method reduce la duplicación al centralizar los pasos estables de un algoritmo y permitir que las subclases definan las partes variables. El ejemplo de café y té muestra que dos procesos pueden producir la misma salida que una solución duplicada, pero con una estructura más fácil de extender y mantener. Su principal costo es la rigidez relativa de la herencia: se debe usar cuando el esqueleto del proceso sea estable.

Fuentes

• Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
• Refactoring.Guru. “Template Method”. https://refactoring.guru/design-patterns/template-method
• Oracle. The Java Tutorials: Object-Oriented Programming Concepts. https://docs.oracle.com/javase/tutorial/java/concepts/